NET2008 DevOps: Practical Assessment 2

Fleet Compliance Auditor | Python, data formats and Git | 5% of final grade | 50 marks repository + 50 marks live demo

The situation

Your team keeps an inventory of network devices as an XML file and a compliance policy as a YAML file. Nobody checks one against the other, so out-of-date devices and unowned devices go unnoticed. Build a command-line tool that reads both files, finds every device that breaks the policy, and writes an audit report as JSON. You decide how to build it: how many functions, which classes, how to organise the code. Everything from Weeks 2 to 6 is available to you. Work in a GitHub Codespace or in a real terminal; your commands must work on Windows, Linux and macOS.

What you build and how you submit

One Python file, **pa2.py**, started like this:

```
python pa2.py DEVICES.xml POLICY.yaml
```

It reads the two files, writes **audit.json** in the current folder, and prints a short summary of your choice. Start from an empty file. There is no template. You keep your work in a **public GitHub repository named net2008-pa2** (Week 6) and submit only the repository URL, as text, in **Practical Assessment 2 Submission**. The root of the repository contains **pa2.py**, **README.md** (how to run it, 10 lines or fewer) and **.gitignore**.

How this assessment is marked

The work is split into three tiers. **Tier 1** is the foundation: the report itself. Every student should be able to finish it. **Tier 2** adds the violation rules. **Tier 3** adds the hard parts: the version comparison and the command-line behaviour. Finish the tiers in order; a partial solution earns partial marks.

Half of the grade is a live demo. After you submit, you show your repository in class: you run your code, explain what it does, and make a small change to it. The date and format are announced later. You need to understand every part of the code you submit, so write code you can explain.

The inventory (XML)

The root element is `inventory` with an optional attribute `site`. It contains `device` elements. A device has attributes `id`, `role` and `os`, an optional `owner` child element holding text, and zero or more `interface` children with attributes `name` and `status`.

- A device with no `id` attribute is skipped and counted in `skipped_devices`.
- An interface `status` is `up` or `down`; any other value (for example `admin-down`) counts as `other`.
- An `owner` is empty or missing when the element is absent or contains only spaces.

The policy (YAML)

A YAML mapping with up to four keys. A key that is missing switches that check off.

Key	Meaning
`min_os`	Mapping of role to the oldest allowed OS version, written as a quoted string. A role that is not a key here is not allowed.
`require_owner`	If true, every device needs a non-empty owner.
`max_down_interfaces`	A device with more than this many interfaces whose status is `down` breaks the policy.

Versions are numbers, not text. Compare them part by part: 15.10 is newer than 15.2, and 9.1.0 equals 9.1. Every version in the test files is whole numbers separated by dots.

What audit.json must contain

Key	Value
`site`	The site attribute of the inventory, or null if it is missing.
`total_devices`	Number of devices kept (not skipped).
`skipped_devices`	Number of devices skipped as described above.
`roles`	Object: role to the number of kept devices with that role.
`interfaces`	Object with keys up, down and other: the number of interfaces of kept devices in each group.
`violations`	List of violation objects (next section). Order does not matter.
`compliant`	Sorted list of the ids of kept devices that have no violation.

Violations

Each violation has `device` (the device id) and `type`. A device can have several. A device whose role is not allowed gets `unknown_role` and no OS check. Checks are only made for keys that are in the policy.

Tier 2 is unknown_role, missing_owner and too_many_down. **Tier 3** is os_outdated (the version comparison) and the command-line behaviour section below. **Tier 1** is the report itself.

type	A device gets it when	Extra fields
`unknown_role`	its role is not a key of min_os	`role` (the role)
`os_outdated`	its version is lower than min_os for its role	`found`, `required` (both as written)
`missing_owner`	require_owner is true and it has no owner	none
`too_many_down`	more than max_down_interfaces interfaces are down	`down` (a whole number)

Example

Files **PA2_sample.xml** and **PA2_policy.yaml** are the example below, and are posted with this brief.

```
<inventory site="ott-dc1">
  <device id="sw-core-01" role="switch" os="15.10">
    <owner>netops</owner>
    <interface name="Gi0/1" status="up"/>
    <interface name="Gi0/2" status="up"/>
    <interface name="Gi0/3" status="down"/>
  </device>
  <device id="sw-edge-02" role="switch" os="15.2">
    <interface name="Gi0/1" status="up"/>
    <interface name="Gi0/2" status="down"/>
  </device>
  <device id="rt-wan-01" role="router" os="17.3">
    <owner>netops</owner>
    <interface name="Te0/0" status="up"/>
    <interface name="Te0/1" status="down"/>
    <interface name="Te0/2" status="down"/>
    <interface name="Te0/3" status="admin-down"/>
  </device>
  <device id="fw-dmz-01" role="firewall" os="9.1.0">
    <owner>secops</owner>
    <interface name="eth1" status="up"/>
  </device>
  <device id="lb-web-01" role="load-balancer" os="4.2">
```

```
    <owner>netops</owner>
    <interface name="eth0" status="up"/>
  </device>
  <device role="switch" os="15.2">
    <owner>netops</owner>
  </device>
</inventory>
```

```
# Audit policy for NET2008 PA2. The version numbers are quoted on purpose.
min_os:
  switch: "15.2"
  router: "17.3.1"
  firewall: "9.1"
require_owner: true
max_down_interfaces: 1
```

A correct audit.json for this input (layout and spacing may differ). Look at why sw-core-01 is compliant, why the device without an id is skipped, and why rt-wan-01 appears twice:

```
{
  "site": "ott-dc1", "total_devices": 5, "skipped_devices": 1,
  "roles": {"switch": 2, "router": 1, "firewall": 1, "load-balancer": 1},
  "interfaces": {"up": 6, "down": 4, "other": 1},
  "violations": [
    {"device": "sw-edge-02", "type": "missing_owner"},
    {"device": "rt-wan-01", "type": "os_outdated", "found": "17.3", "required": "17.3.1"},
    {"device": "rt-wan-01", "type": "too_many_down", "down": 2},
    {"device": "lb-web-01", "type": "unknown_role", "role": "load-balancer"}
  ],
  "compliant": ["fw-dmz-01", "sw-core-01"]
}
```

Behaviour

- A normal run exits with code 0, even when there are violations.
- If either file does not exist, or is not valid XML or YAML, or the policy is not a mapping: print a message that includes the file name and exit with code 1. No traceback.
- If the program is not given exactly two arguments: print a line starting with `usage:` and exit with code 2.
- An inventory with no devices, or a policy that is an empty file, must work.

Show these techniques

Your code must show the following techniques, and your design should include them naturally:

- A custom exception class that you raise and catch (for example for a bad input file).
- At least one class of your own (for example `Device`) with methods that you use.
- File handling with `with` or pathlib, and no bare `except:`.
- Clean code: only the standard library plus **PyYAML** (`import yaml`), no `eval`/`exec`/`global`, nothing running at import except definitions and an `if __name__ == "__main__":` block, and a docstring on at least 80% of your functions and classes.
- Use `yaml.safe_load`, never `yaml.load`: YAML can run code when loaded the unsafe way.

Git and GitHub

Your repository is marked from a fresh clone, so the history must show real work, as in the Week 6 lab:

- At least 5 commits, each with a message that says what changed (not `update` or `asdf`).
- At least one feature branch that you created, worked on and merged into `main`.
- A `.gitignore` that keeps `audit.json`, `__pycache__` and virtual environments out of the repository.

- The repository is public and you do not push after the deadline.

Marks

Repository (submitted by the deadline)	Marks
Tier 1: runs and writes a valid audit.json 3, site and totals 3, roles 4, interfaces 4	14
Tier 2: violations unknown_role 4, missing_owner 4, too_many_down 4	12
Tier 3: os_outdated with correct version comparison 6, command-line behaviour and error handling 6	12
The four techniques listed above (1.5 each)	6
Git and GitHub, as described above (1.5 each)	6
Repository total	50
Live demo in class: you explain your code, and change it	50
Total	100

Rules

- Work alone and write your own code. Sharing or copying code is an academic integrity offence.
- Put your name and student ID in a comment at the top of pa2.py and in README.md.
- The deadline is **Saturday, October 31 at 11:59 PM**. Late submissions receive a grade of 0.
- Your program will be run on other inventories and policies, including edge cases. Do not tune your code to the example.

Where to start

Create the repository and make a first commit. Parse the XML and print the device ids. Count roles and interfaces. That is Tier 1; commit it. Then add one violation at a time, committing after each (Tier 2). Do the version comparison last, and test it with 15.10 against 15.2 (Tier 3).